SISTEMAS OPERATIVOS MONOPUESTO (SMR 1.º)

Ampliación: instalar Arch Linux sobre un RAID 0 por software

Tipo Explicación | **Nivel** 3 · Avanzado | **Tema** 14 | **CM** CM2 | **Herramientas** H1 (VirtualBox), H4 (terminal Linux) | **Carácter** ACTIVIDAD OPCIONAL DE AMPLIACIÓN

> 🚀 **Actividad OPCIONAL.** No es obligatoria para aprobar el módulo. Se presenta como valor añadido en la defensa del reto final. Se evalúa **aparte**.
>
> Recuerda que esta instalación **no forma parte** del itinerario obligatorio. Esto es un extra para quien quiera ir más allá.

Índice

- Ampliación: instalar Arch Linux sobre un RAID 0 por software
  - Índice
  - Antes de empezar: qué es y qué no es un RAID 0
  - 1. Preparar la máquina virtual
  - 2. Arranque y comprobaciones
  - 3. Particionar los dos discos
  - 4. Crear los arrays con mdadm
  - 5. Formatear y montar
  - 6. Sistema base, fstab y mdadm.conf
  - 7. Configuración dentro del chroot
  - 8. mkinitcpio: el paso que no se puede saltar
  - 9. GRUB en UEFI
    - Sobre duplicar la partición EFI en el segundo disco
  - 10. Primer arranque y verificación del array
  - 11. Experimento: qué pasa si falla un disco
  - Problemas frecuentes

Antes de empezar: qué es y qué no es un RAID 0

RAID 0 reparte los datos entre dos discos (*striping*). Cada bloque de información se escribe alternando entre los dos, así que se leen y se escriben en paralelo.

	RAID 0
Capacidad útil	La suma de los dos discos (100 %)
Rendimiento	Aproximadamente el doble en lectura y escritura secuencial
Tolerancia a fallos	**Ninguna**
Si falla un disco	**Se pierde todo**, también lo que estaba en el disco sano

> ⚠️ **Ojo con la palabra «redundante».** RAID significa *Redundant Array of Independent Disks*, pero **el nivel 0 no tiene ninguna redundancia**. Al contrario: con dos discos en RAID 0 la probabilidad de perderlo todo es **el doble** que con un disco solo, porque basta con que falle cualquiera de los dos.
>
> **Analogía.** RAID 0 es partir un documento en dos mitades y guardar cada mitad en un cajón distinto. Se lee más rápido porque dos personas pueden buscar a la vez, pero si se pierde un cajón el documento entero es ilegible. RAID 1 sería fotocopiarlo y guardar una copia en cada cajón: ocupa el doble, pero se puede perder un cajón entero sin consecuencias.

Entonces, ¿para qué sirve? Para datos que se pueden regenerar y donde manda la velocidad: cachés, archivos temporales de edición de vídeo, sistemas de compilación. **Nunca** para datos que no puedas volver a crear, y **nunca** sin copias de seguridad aparte.

> ℹ️ **Contexto en la asignatura.** En el itinerario obligatorio se practica un **volumen reflejado (RAID 1) en la máquina de Windows** con simulación de fallo de un disco: ahí es donde se trabaja la redundancia de verdad. Esta ampliación practica `mdadm` **y el arranque desde un array**, que es lo que la hace interesante técnicamente. En el entregable hay que dejar clara esta diferencia.

1. Preparar la máquina virtual

Recurso	Valor
Nombre	`Arch-RAID0`
Memoria base	2048 MB

Recurso	Valor
Procesadores	2
Habilitar EFI	**Sí**
Discos	**Dos** discos de **12 GB** cada uno, reserva dinámica
Red	NAT

Para añadir el segundo disco: máquina apagada → **Configuración** → **Almacenamiento** → controlador SATA → icono de añadir disco duro → **Crear** → 12 GB.

Esquema que vamos a montar:



> ❗ **La partición EFI no entra en el RAID.** El firmware UEFI lee la partición EFI **antes** de que exista ningún sistema operativo, así que no sabe nada de `mdadm` ni de arrays. Tiene que ser una partición FAT32 normal y corriente. Por eso `sda1` y `sdb1` quedan fuera.

2. Arranque y comprobaciones

Arranca desde la ISO de Arch y:

```
loadkeys es
setfont ter-132b                          # opcional, letra más grande
cat /sys/firmware/efi/fw_platform_size    # debe responder 64
timedatectl set-ntp true
ping -c 3 archlinux.org
lsblk
```

Debes ver **dos** discos de 12 GB: `sda` y `sdb`.

3. Particionar los dos discos

Usamos `sgdisk`, que permite particionar desde la línea de comandos sin menús — útil porque hay que hacer exactamente lo mismo en los dos discos.

```
sgdisk -n 1:0:+512M -t 1:ef00 -c 1:"EFI System"  /dev/sda
sgdisk -n 2:0:+9G   -t 2:fd00 -c 2:"Linux RAID"  /dev/sda
sgdisk -n 3:0:0     -t 3:fd00 -c 3:"Linux RAID"  /dev/sda
```

Parte	Significado
`-n 2:0:+9G`	Nueva partición nº 2, desde el primer hueco libre, de 9 GB
`-t 2:fd00`	Tipo `fd00` = *Linux RAID*
`-t 1:ef00`	Tipo `ef00` = *EFI System*
`-n 3:0:0`	Desde el primer hueco hasta el final del disco
`-c n:"..."`	Nombre descriptivo de la partición

Copiar exactamente la misma tabla al segundo disco:

```
sgdisk /dev/sda -R /dev/sdb -G
```

Opción	Qué hace
`-R /dev/sdb`	Replica la tabla de `sda` en `sdb`
`-G`	Genera **nuevos GUID**. Imprescindible: dos discos con el mismo identificador confunden al sistema

Comprueba:

```
lsblk
sgdisk -p /dev/sda
sgdisk -p /dev/sdb
```

> ℹ️ Si te equivocas y quieres empezar de cero: `sgdisk -Z /dev/sda` borra la tabla de particiones.

4. Crear los arrays con mdadm

`mdadm` (*multiple devices admin*) es la herramienta de RAID por software de Linux.

```
pacman -Sy mdadm
```

```
mdadm --create --verbose /dev/md0 --level=0 --raid-devices=2 /dev/sda2 /dev/sdb2
mdadm --create --verbose /dev/md1 --level=0 --raid-devices=2 /dev/sda3 /dev/sdb3
```

Forma abreviada, equivalente:

```
mdadm -Cv /dev/md0 -l0 -n2 /dev/sd[a-b]2
mdadm -Cv /dev/md1 -l0 -n2 /dev/sd[a-b]3
```

Parámetro	Significado
`--create /dev/md0`	Crear el dispositivo virtual `md0`
`--level=0`	Nivel RAID 0
`--raid-devices=2`	Con dos discos
`/dev/sd[a-b]2`	Comodín que se expande a `/dev/sda2 /dev/sdb2`

Comprueba:

```
lsblk
cat /proc/mdstat
mdadm --detail /dev/md0
```

`/proc/mdstat` es el estado en vivo de todos los arrays. `md0` debe aparecer como `active raid0` con unos 18 GB.

5. Formatear y montar

> ℹ️ **Simplificación respecto a la guía del curso anterior.** Aquella creaba una tabla de particiones **dentro** de cada array (`/dev/md0p1`). No hace falta y complica el `fstab` y el arranque: el array ya es un dispositivo de bloques y se puede formatear directamente. Aquí formateamos `/dev/md0` y `/dev/md1` sin particionarlos.

```
mkfs.ext4 /dev/md0            # raíz, sobre el array de 18 GB
mkswap    /dev/md1            # swap, sobre el array de ~5 GB
swapon    /dev/md1
mkfs.fat -F32 /dev/sda1       # EFI: partición normal, fuera del RAID
```

Montaje, **primero la raíz**:

```
mount /dev/md0 /mnt
mkdir -p /mnt/boot/efi
mount /dev/sda1 /mnt/boot/efi
```

Comprobación:

```
lsblk -f
findmnt /mnt
```

6. Sistema base, fstab y mdadm.conf

```
pacstrap -K /mnt base linux linux-firmware \
  mdadm networkmanager grub efibootmgr \
  sudo nano vim base-devel man-db
```

> ℹ️ `mdadm` tiene que estar instalado **dentro** del sistema nuevo, no sólo en el instalador. Si se te olvida, el sistema no podrá ensamblar el array al arrancar.

```
genfstab -U /mnt >> /mnt/etc/fstab
cat /mnt/etc/fstab
```

Guardar la configuración del array:

```
mdadm --detail --scan >> /mnt/etc/mdadm.conf
cat /mnt/etc/mdadm.conf
```

> ⚠️ **Corrección respecto a la guía del curso anterior.** Allí escribía en `/mnt/etc/mdadm-conf`, con guion. El fichero que Linux lee se llama **`mdadm.conf`**, con punto. Con el nombre mal, el array puede montarse igualmente por autodetección… o no, y el sistema no arranca sin que se entienda por qué. Comprueba el nombre.

El fichero debe contener una línea `ARRAY` por cada array, con su UUID:

```
ARRAY /dev/md0 metadata=1.2 name=archiso:0 UUID=xxxxxxxx:xxxxxxxx:xxxxxxxx:xxxxxxxx
ARRAY /dev/md1 metadata=1.2 name=archiso:1 UUID=xxxxxxxx:xxxxxxxx:xxxxxxxx:xxxxxxxx
```

```
arch-chroot /mnt
```

7. Configuración dentro del chroot

Igual que en la guía de instalación normal, resumido:

```
ln -sf /usr/share/zoneinfo/Europe/Madrid /etc/localtime
hwclock --systohc

nano /etc/locale.gen        # descomentar es_ES.UTF-8 y en_US.UTF-8
locale-gen
echo "LANG=es_ES.UTF-8" > /etc/locale.conf
echo "KEYMAP=es" > /etc/vconsole.conf

echo "arch-raid" > /etc/hostname
nano /etc/hosts
```

Contenido de `/etc/hosts`:

```
127.0.0.1   localhost
::1         localhost
127.0.1.1   arch-raid.localdomain   arch-raid
```

> ⚠️ **Otra corrección.** La guía anterior usaba tres `echo ... > /etc/hosts` seguidos. Con `>` cada línea borra la anterior, así que sólo quedaba la última. Edita el fichero, o usa `>>`.

Usuarios:

```
passwd
useradd -m -G wheel -s /bin/bash alumno
passwd alumno
EDITOR=nano visudo    # descomentar %wheel ALL=(ALL:ALL) ALL
```

> ℹ️ Usa `visudo` y no `nano /etc/sudoers`: valida la sintaxis antes de guardar. Un error en ese fichero deja el sistema sin `sudo` para nadie.

8. mkinitcpio: el paso que no se puede saltar

Éste es el paso que diferencia esta instalación de la normal, y donde falla todo el mundo.

El *initramfs* es un mini-sistema que se carga antes que el sistema real, y es quien tiene que ensamblar el array **antes** de poder montar la raíz. Si no lleva el módulo de RAID, no encuentra `/dev/md0` y el arranque muere con `Cannot open root device`.

```
nano /etc/mkinitcpio.conf
```

Busca la línea `HOOKS=` y añade **`mdadm_udev`** justo después de `block`:

```
HOOKS=(base udev autodetect microcode modconf kms keyboard keymap consolefont block mdadm_udev
filesystems fsck)
```

Y regenera la imagen:

```
mkinitcpio -P
```

> 🧩 **Analogía.** El initramfs es el conserje que abre el edificio antes de que llegue nadie. Si el conserje no tiene la llave del RAID, se queda en la puerta y nadie entra. `mdadm_udev` es esa llave.

Busca en la salida de `mkinitcpio -P` que no haya errores y que se mencione el módulo `mdadm`.

Opcionalmente, para recibir avisos de fallo del array, revisa la línea `MAILADDR` de `/etc/mdadm.conf`.

9. GRUB en UEFI

```
grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=GRUB
grub-mkconfig -o /boot/grub/grub.cfg
systemctl enable NetworkManager
```

Sobre duplicar la partición EFI en el segundo disco

La guía anterior copiaba la partición EFI al segundo disco y la registraba en el firmware:

```
# umount /dev/sda1 ; dd if=/dev/sda1 of=/dev/sdb1
# efibootmgr -c -g -d /dev/sdb -p 1 -L "ArchLinux #2" -l "\EFI\GRUB\grubx64.efi"
```

> ⚠️ **En RAID 0 esto no sirve de nada, y conviene entender por qué.** Duplicar el arranque en el segundo disco protege del caso «se rompe el disco 1 pero el sistema sigue estando entero en el disco 2». Eso ocurre en **RAID 1**. En **RAID 0**, si se rompe el disco 1 la mitad de los datos del sistema desaparece: podrías arrancar el GRUB del segundo disco, pero no habría sistema al que arrancar. Es como tener una segunda llave de una casa que ya no existe.
>
> Además el comando original tenía un error: ponía `-d /dev/sdb1`. El parámetro `-d` espera el **disco** (`/dev/sdb`) y el número de partición se indica aparte con `-p 1`.
>
> **Por eso este paso se omite aquí.** Si quieres hacerlo, hazlo como ejercicio de `efibootmgr` y explica en el entregable por qué no aporta tolerancia a fallos en RAID 0. Esa explicación puntúa más que el propio comando.

Salir y reiniciar:

```
exit
umount -R /mnt
swapoff -a
reboot
```

Quita la ISO en cuanto se apague.

10. Primer arranque y verificación del array

Entra como `alumno` y comprueba:

```
cat /proc/mdstat
```

```
Personalities : [raid0]
md0 : active raid0 sda2[0] sdb2[1]
      18855936 blocks super 1.2 512k chunks

md1 : active raid0 sda3[0] sdb3[1]
      4188160 blocks super 1.2 512k chunks
```

```
lsblk
df -h /
sudo mdadm --detail /dev/md0
free -h
findmnt /
```

Medición de rendimiento (parte del entregable). Compárala con la máquina de Arch normal:

```
sudo pacman -S hdparm

# Escritura secuencial
dd if=/dev/zero of=/tmp/prueba bs=1M count=1024 conv=fdatasync status=progress

# Lectura
sudo hdparm -Tt /dev/md0
sudo hdparm -Tt /dev/sda
rm /tmp/prueba
```

> ⚠️ **Aviso de honestidad experimental.** Los dos discos virtuales están en el **mismo disco físico** de tu ordenador, así que la mejora de RAID 0 será mucho menor de lo teórico, y puede incluso ser negativa. Eso **no invalida la práctica**: al contrario, explicar por qué el resultado no es el esperado es exactamente lo que se valora. Un RAID por software sobre un solo disco físico no acelera nada; el cuello de botella sigue siendo el mismo cabezal.

11. Experimento: qué pasa si falla un disco

Con una **instantánea hecha antes**, para poder volver:

1. Apaga la máquina y crea la instantánea `raid-ok`.
2. En **Configuración** → **Almacenamiento**, quita el segundo disco (`sdb`) sin borrarlo.
3. Arranca.

Anota **exactamente** lo que ocurre: qué mensaje sale, en qué momento del arranque, si llegas a alguna consola de emergencia. Compáralo con lo que ocurriría en un RAID 1.

4. Vuelve a añadir el disco y comprueba con `cat /proc/mdstat` si el array se recupera solo.
5. Restaura la instantánea si hace falta.

> ℹ️ Esto es lo más valioso de toda la ampliación: **haber visto fallar el array con tus ojos**. En la defensa del reto, poder describir el mensaje exacto que sale vale más que haber completado la instalación.

Problemas frecuentes

Síntoma	Causa	Solución
`Cannot open root device` / cae a la consola de emergencia	Falta `mdadm_udev` en los HOOKS	Rescate con chroot, editar `/etc/mkinitcpio.conf` y `mkinitcpio -P`
El array no aparece al arrancar	`mdadm.conf` mal escrito o `mdadm` no instalado en el sistema	Comprobar el nombre del fichero y `pacman -S mdadm`
`grub-install` falla con `efivarfs`	La MV no tiene EFI activado	Apagar y activar EFI
`mdadm: device or resource busy`	Restos de un array anterior	`mdadm --stop /dev/md0` y `mdadm --zero-superblock /dev/sdX2`
Los dos discos tienen el mismo GUID	Se copió la tabla sin `-G`	`sgdisk /dev/sda -R /dev/sdb -G` otra vez
`/dev/md127` en lugar de `/dev/md0`	El nombre del array no está fijado	Es normal si falta `mdadm.conf`; añadirlo y regenerar el initramfs

Rescate con chroot (para arreglar los HOOKS desde la ISO):

```
loadkeys es
mdadm --assemble --scan
lsblk
mount /dev/md0 /mnt
mount /dev/sda1 /mnt/boot/efi
arch-chroot /mnt
# ... arreglar ...
mkinitcpio -P
exit
umount -R /mnt
reboot
```